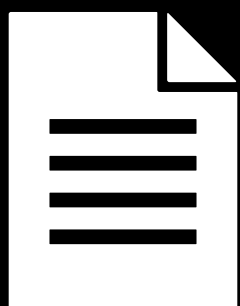


완전 초보자를 위한 버전 관리

Git? GitHub!

텍스트 파일이 하나 있다.



memo.txt

수정된 내용을 기록해두고 싶다! 어떻게 할까?



memo.txt

수정할 때 마다 다른 파일로 저장



memo.txt

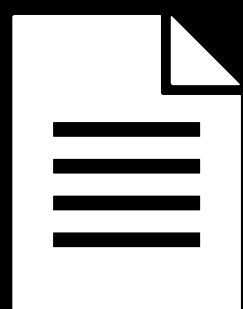


memo1.txt

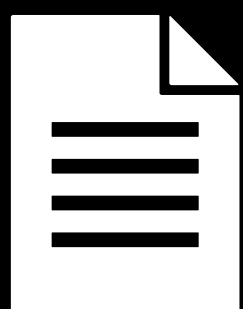


memo2.txt

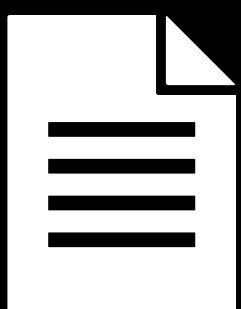
시간이 지나면... 흠.....



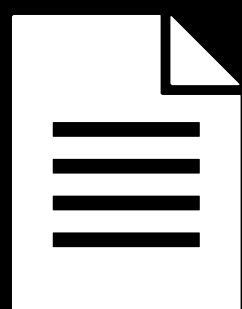
memo.txt



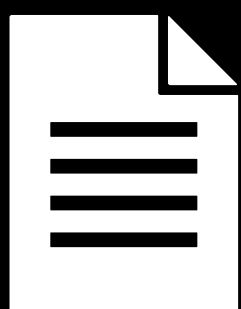
memo1.txt



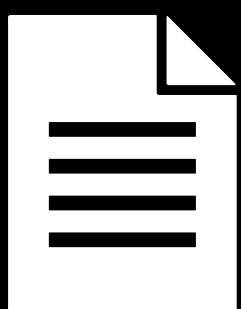
memo2.txt



memo3.txt



memo4.txt



memo5.txt

파일이 하나가 아니라면?



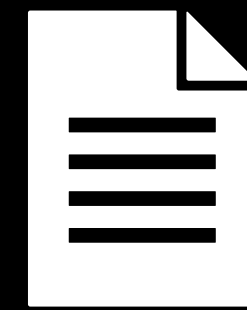
memo.txt



memo1.txt



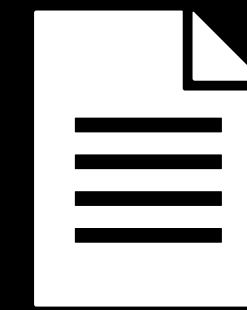
memo2.txt



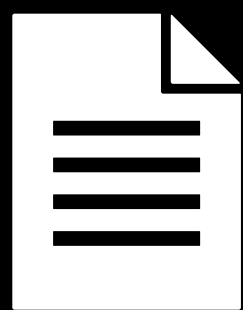
memo3.txt



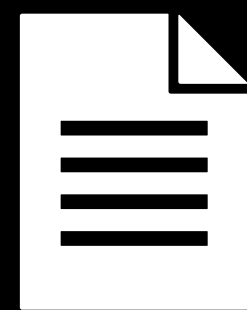
memo4.txt



memo5.txt



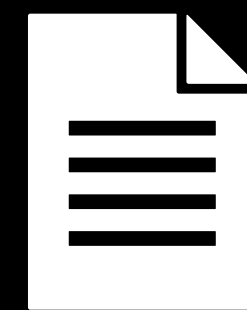
todo.txt



todo1.txt



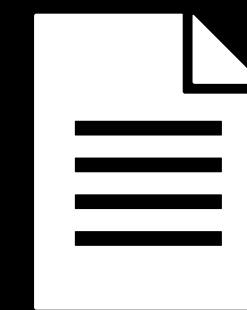
todo2.txt



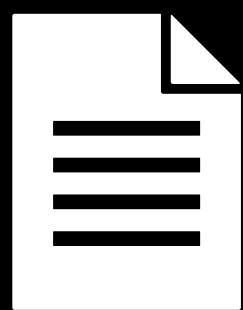
todo3.txt



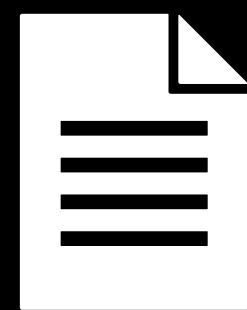
todo4.txt



todo5.txt



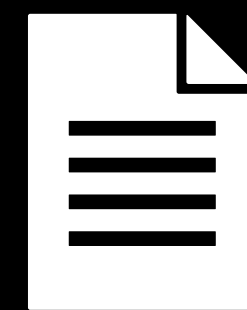
test.txt



test1.txt



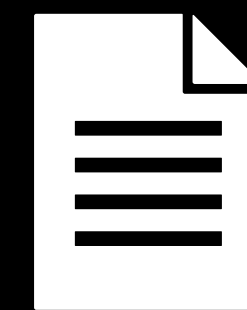
test2.txt



test3.txt



test4.txt



test5.txt

OTL

수정하는 사람이 여러명이면?



memo.txt



memo1.txt



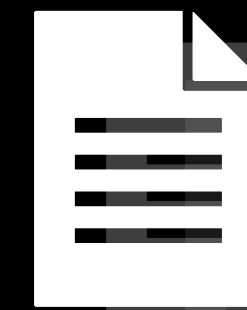
memo2.txt



memo3.txt



memo4.txt



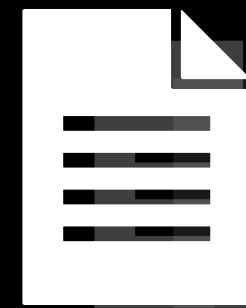
memo5.txt



todo.txt



todo1.txt



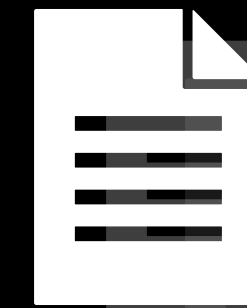
todo2.txt



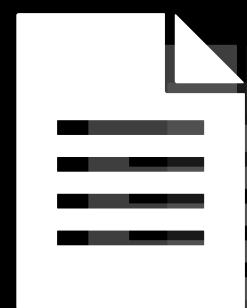
todo3.txt



todo4.txt



todo5.txt



test.txt



test1.txt



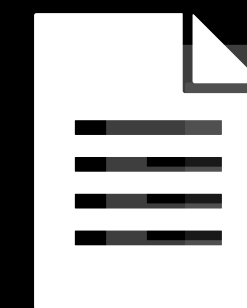
test2.txt



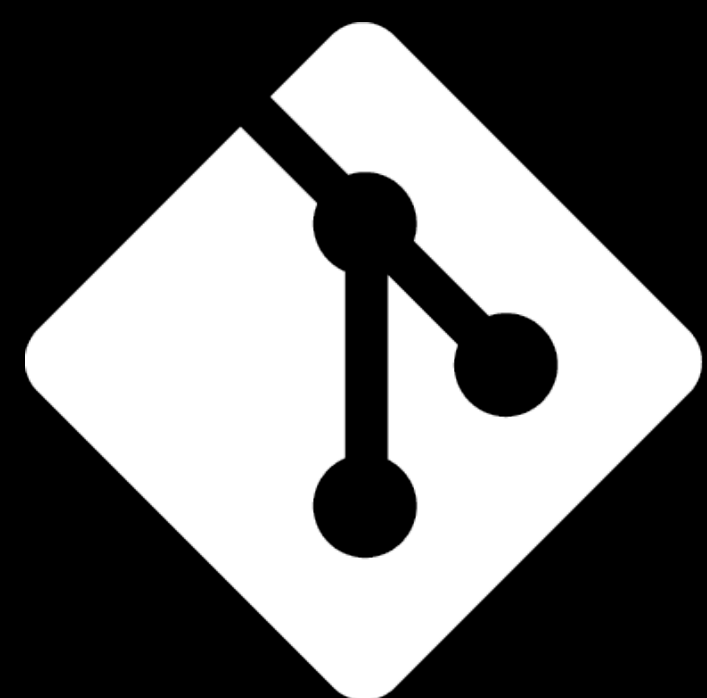
test3.txt



test4.txt



test5.txt



git

Git?

Version Control System

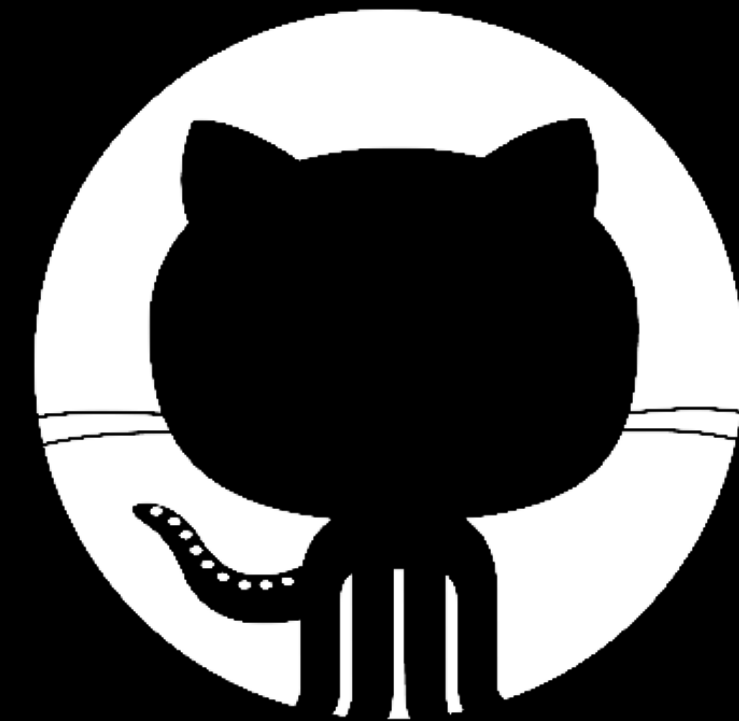


- 코드 저장소
- 이력을 관리하는 저장소
- Code를 저장 및 저장지점으로 되돌아갈 수 있도록 해주는 시스템
- 분산 버전 관리 시스템
 - 여러 개발 PC와 저장소에 분산해서 저장

GitHub?

코드저장소(Git)을 웹에 옮겨 놓은 원격저장소

- 웹에 있을 때 장점?
 - 공유 가능(협업)
 - 코드 복구 가능



분산버전관리?

- 버전 관리란?

- 파일 변화를 시간에 따라 기록했다 특정 시점의 버전을 다시 가져올 수 있는 것

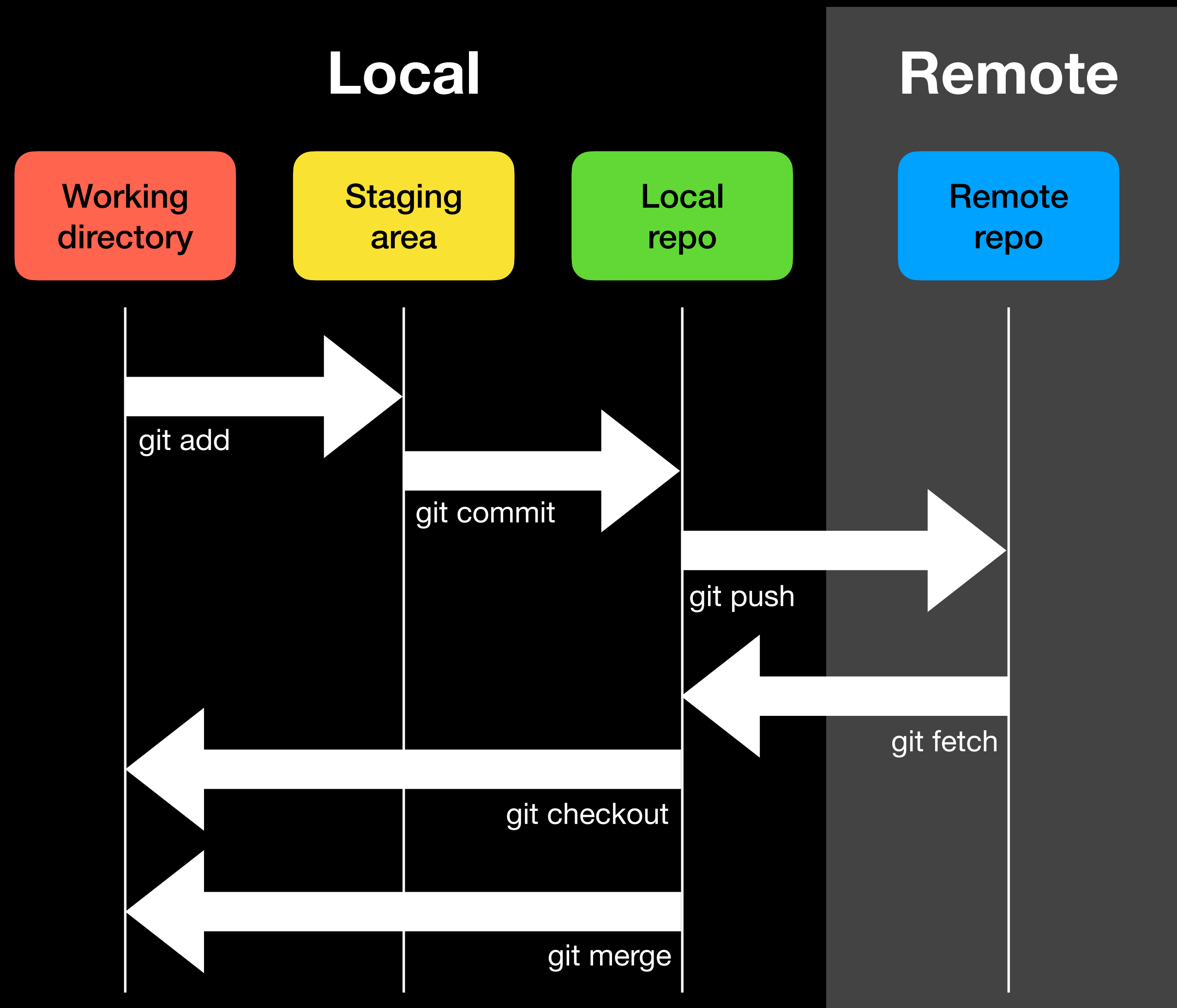
- 장점

- 소스 코드가 변경된 이력을 쉽게 확인 가능
 - 특정 시점에 저장된 버전과 비교 가능
 - 특정 시점으로 되돌아갈 수 있음
 - 내가 올리려는 파일이 누군가 편집한 내용과 충돌시 덮어쓰기 위험에서 벗어나 해결 가능

Repository

- **Remote Repository** (원격 저장소)
 - 파일이 원격 저장소 전용 **서버에서 관리** 되며 여러 사람이 함께 공유하기 위한 저장소
- **Local Repository** (로컬 저장소)
 - 내 PC에 파일이 저장되는 **개인** 전용 저장소

Repository



- **Working directory**
 - 로컬 작업 디렉토리
- **Staging area**
 - commit 시 반영되는 파일 보관
- **Local repo**
 - 로컬에 저장된 파일을 push할 경우 원격 저장소로 반영
- **Remote repo**
 - 원격 저장소

Repository 생성

실습 with GitHub

- 1. GitHub web page → new repository
- 2. 생성된 주소 복사
- 3. Git bash로 config 초기 설정 (git bash 없다면 설치하기)
 - `git config --global user.name "your_github_name"`
 - `git config --global user.email "your_github_email"`
- 4. Git 초기화

Git 명령어

Add

- 저장소에 파일 추가 (Staged)
- 경우 1 - 디렉토리 안 모든 파일 추가 : `git add .`
- 경우 2 - 특정 파일만 추가 : `git add [path/filename]`
 - Ex) `git add memo.txt`

Git 명령어

Commit

- Staging 한 파일 커밋
- 커밋 메시지는 해당 작업에서 추가/변경된 사항 간략히 작성
- 타인이 알아볼 수 있도록 작성 및 영어로 작성하는 것이 좋다.
- `git commit -m '커밋 메시지'`

Git 명령어

Push

- 원격 저장소로 Push
- `git push [원격 저장소 이름] [원격 저장소 브랜치 이름]`
 - Ex) `git push origin master`
 - → 로컬 저장소의 변경사항을 원격 저장소(origin)의 브랜치(master)에 반영

Branch

- 각자 독립적인 작업 영역(저장소)
- 즉, 여러 사람이 동일한 소스코드를 기반으로 서로 다른 작업
- 분리된 작업 영역에서 마음대로 소스코드 변경
- 작업이 끝난 사람은 메인 브랜치에 자신의 브랜치의 변경사항을 적용

Git 명령어

Branch

- 새로운 Branch 생성
- `git branch [브랜치 이름]`
 - Ex) `git branch develop`
 - → develop이라는 이름의 새로운 브랜치 생성

Git 명령어

Checkout

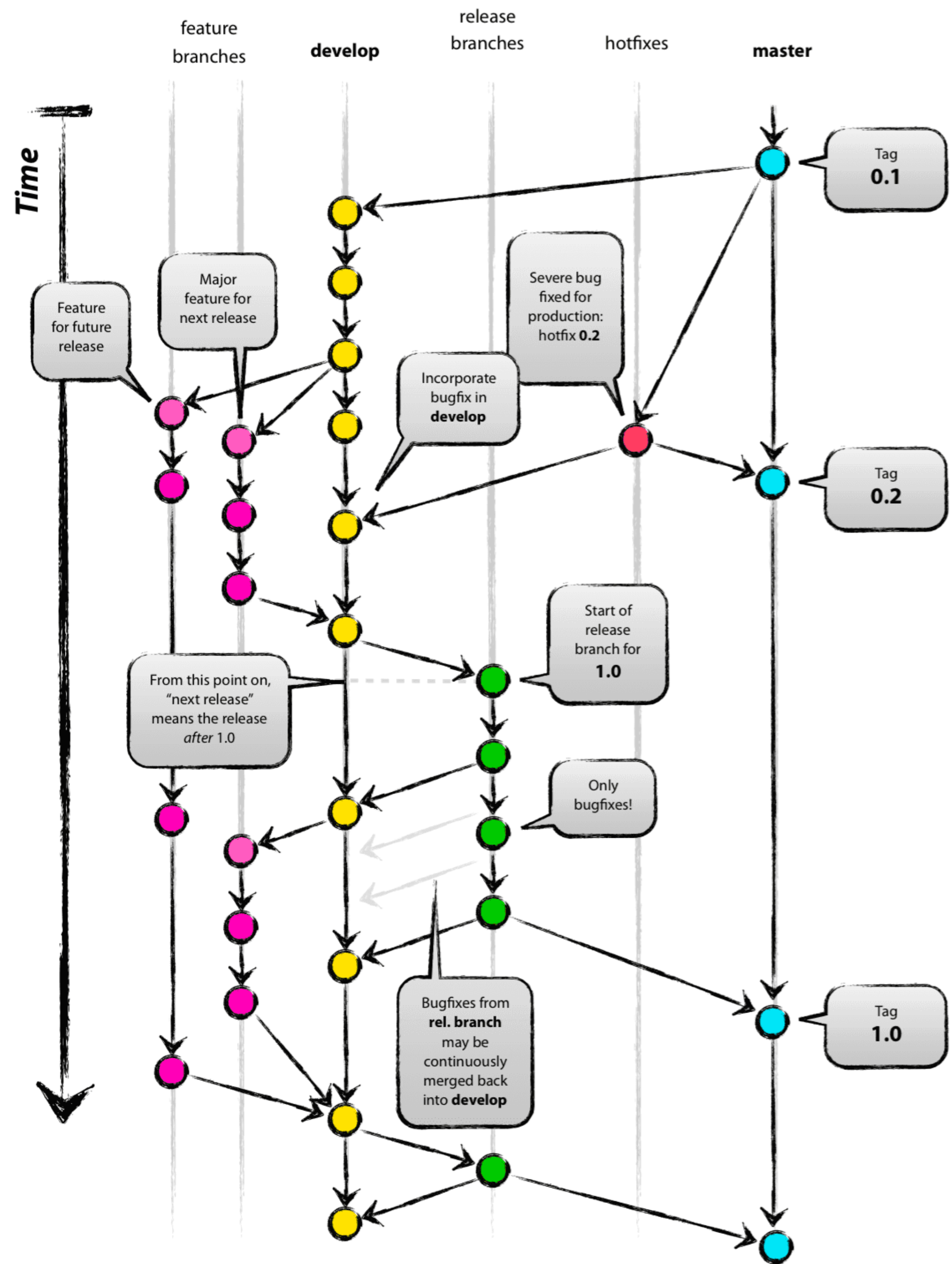
- 브랜치로 이동
- `git checkout [브랜치 이름]`
 - Ex) `git checkout develop`
 - → develop 브랜치로 이동

Git 명령어

Merge

- 다른 Branch에 있는 내용을 병합
- `git merge [브랜치 이름]`
 - Ex) `git merge develop`
 - → develop 브랜치를 main과 병합 (HEAD가 main일 때)
 - 또는 `git merge main develop`

Git Flow



그밖에 알아두면 좋은 키워드

- git clone
- git pull
- git tag
- git rebase
- git statsh
- GitHub Desktop
- .gitignore
- README.md

수고했어요 ㄴ (≡ ∇ ≡*)o

- 끝!



참고하면 좋은 사이트

- <https://backlog.com/git-tutorial/kr/>
- <https://www.pigno.se/barn/tutorial-git/docs/#/>
- <https://opentutorials.org/module/3733>